CHAINSECURITY

PUBLIC

Code Assessment
of the Aave v3.8 Pool Actions Reentrancy Guard Smart Contracts

October 27, 2026

Produced for



Contents

1	Executive Summary	3
2	Assessment Overview	4
3	System Overview	5
4	Trust Model	6
5	System Considerations	7
6	Terminology	8
7	Findings	9
8	Limitations and use of report	11

1 Executive Summary

Dear Aave Labs Team,

Thank you for trusting ChainSecurity with the review of the reentrancy guard added to Aave's `Pool` contract as part of the Aave v3.8 upgrade.

As a globally trusted security partner for tier-one DeFi applications, ChainSecurity provides specialized, independent technical analysis for complex smart contract systems.

We greatly appreciate the opportunity to collaborate with Aave Labs, a leading actor in the DeFi space, on this review.

Aave Labs implements the Aave V3 lending protocol, allowing users to supply, withdraw, borrow, repay, liquidate, and flash-loan assets through a central `Pool` contract.

We have conducted a thorough review of reentrancy protection across all state-mutating `Pool` actions. We identified one informational issue: `eliminateReserveDeficit` was not reentrancy guarded, see **`eliminateReserveDeficit` Is Not Reentrancy Guarded**. We found no way to turn it into an exploitable issue, and it has since been addressed in **Version 2**. No issues remain unaddressed. We also assessed the gas overhead the guard adds, flagging that integrations relying on tight gas estimates may need to adjust, see **Added Reentrancy-Guard Gas Cost Can Break Callers Using Tight Gas Estimates**.

Our review confirmed that:

- No legitimate action is blocked by the new guard. For example, moving aTokens during a liquidation uses `transferOnLiquidation()`, which does not invoke the now-guarded `finalizeTransfer()` hook.
- We verified that a malicious reentrant token used in a flashloan cannot corrupt the repayment and premium accounting of other assets, since flashloans intentionally do not acquire the guard, relying only on `onlyNotReentrant` to preserve composability.

This is the first stage of the audit, scoped to the reentrancy guard. We will continue reviewing the rest of the V3 codebase. In its current state (**Version 2**), the guard itself raises no concerns. We ask Aave Labs to carefully review **System Overview**, **Trust Model** and **System Considerations**.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the Aave v3.8 Pool Actions Reentrancy Guard repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	24 September 2026	fee66f32fd99c37135ae65b067af936e260f0e8d	Initial review
2	20 October 2026	7739cbcf15270626a2feb8d9843541872ddc9431	Aave v3.8 release candidate

The dates above are the dates on which the code was received. The commits listed were subsequently re-created in the public repository with identical source code under `src/`, so their commit dates are later than the dates received.

For the Solidity smart contracts, the compiler version `0.8.27` was chosen. Starting with **Version 2**, the project's `foundry.toml` targets the `cancun` EVM version (previously `shanghai`), which the transient-storage guard requires.

The review of **Version 1** focuses exclusively on the change between `v3.7` release and **Version 1** commit , introducing the reentrancy guard on `Pool`'s actions. **Version 2** was reviewed as the delta on top of **Version 1**, and, in addition, as a whole against `v3.7`. Code outside these changes is not in scope.

```
src/
└── contracts/
    ├── protocol/
    │   └── pool/
    │       └── Pool.sol
    └── instances/
        └── PoolInstance.sol
```

3 System Overview

At the end of this report section, we have added subsections for each of the changes according to the versions.

This system overview describes the initially received version (**Version 1**) of the contracts as defined in the Assessment Overview.

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Aave Labs offers the Aave V3 lending protocol, allowing users to supply, withdraw, borrow, repay, liquidate, and flash-loan assets through a central `Pool` contract.

This review focuses on a reentrancy-guard fix applied to the `Pool` contract's user-facing actions (e.g. `supply()`, `withdraw()`, `borrow()`, `repay()`, `liquidationCall()`). Previously, these actions were not protected against reentrancy. The fix introduces a guard based on OpenZeppelin's `ReentrancyGuardTransient` (transient-storage backed):

- A `nonReentrant` modifier locks a `Pool` action for its full execution, blocking any nested call into another guarded action.
- A separate `onlyNotReentrant` modifier is applied to the flash-loan entry points (`flashLoan()`, `flashLoanSimple()`): it checks that no guarded action is already in progress, but does not itself hold the lock, since flash loans intentionally call back into the receiver, which may call other guarded `Pool` actions.

3.1 Changes in Version 2

Version 2 is the delta on top of **Version 1**: `Pool.eliminateReserveDeficit()` is now marked `nonReentrant`.

4 Trust Model

4.1 Roles

Aave DAO / Governance

- Trust level: Fully trusted.
- Permissions: Lists and configures reserves (including supply/borrow caps), upgrades contracts, and otherwise administers the protocol.
- Worst-case impact: Could misconfigure or maliciously configure the protocol.

Users

- Trust level: Untrusted.
- Permissions: Supply, withdraw, borrow, repay, liquidate, and flash-loan through the `Pool` using any listed reserve.
- Worst-case impact: May act maliciously (e.g. attempt reentrancy, price manipulation, or griefing) but are restricted to the permissions and economic limits enforced by the protocol.

Umbrella

- Trust level: Fully trusted. The address is registered by the DAO in the `PoolAddressesProvider`.
- Permissions: It is the only address allowed to call `eliminateReserveDeficit()` on `Pool`, which writes off a reserve's recorded deficit (bad debt) by burning Umbrella's own aToken balance for that reserve.
- Worst-case impact: If compromised it could fail to cover real deficits. It cannot access other users' funds through this entry point.

4.2 External Dependencies

Listed tokens

- Assumption: Reserves are expected to be vetted and listed in accordance with the ARFC Technical Asset Listing Framework before being onboarded by the DAO.
- Worst-case impact: Even if a listed token turns out to be malicious or otherwise misbehaves (e.g. rebasing, fee-on-transfer, non-standard return values), the protocol's exposure to it must not exceed the configured supply and borrow caps for that reserve.

Deployment chain

- Assumption: The reentrancy guard on `Pool` is implemented with OpenZeppelin's `ReentrancyGuardTransient`, which relies on EIP-1153 transient storage (`TSTORE`/`TLOAD`, available from the Cancun hardfork). We assume Aave v3.8 Pool Actions Reentrancy Guard is only deployed on chains/rollups whose EVM implementation supports these opcodes.
- Worst-case impact: Deploying on a chain without EIP-1153 support would cause every guarded `Pool` action to revert (invalid opcode), making the Pool entirely unusable.

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC1 Added Reentrancy-Guard Gas Cost Can Break Callers Using Tight Gas Estimates

System Consideration | Version 1

In the Aave v3.8 upgrade, every guarded action in `Pool` now sets and clears a reentrancy-guard flag, adding roughly 550 gas to each call. Historical-transaction replay across three mainnet windows and 14 other chains shows that a measurable share of `Pool`-touching transactions, on average about 0.5% and up to 2% on Base, run out of gas when replayed against the fixed implementation, because the caller supplied a gas limit computed by `eth_estimateGas` with little or no margin and forwarded all remaining gas into the call reaching `Pool`'s guarded logic. The affected callers do not rely on a hardcoded gas stipend and forward whatever gas is available, so the failure disappears once the caller re-estimates gas against the fixed code, but any integration that caches or hardcodes a gas limit for these calls instead of re-estimating per transaction will see its Pool interactions revert with an out-of-gas error after the guard is deployed.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

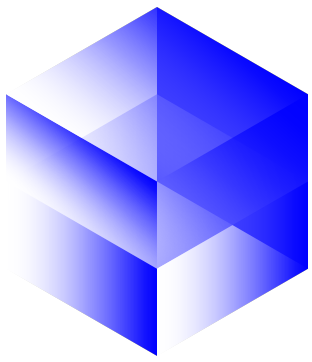
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	**Impact**		
	High	Medium	Low
High	**Critical**	**High**	**Medium**
Medium	**High**	**Medium**	**Low**
Low	**Medium**	**Low**	**Low**

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

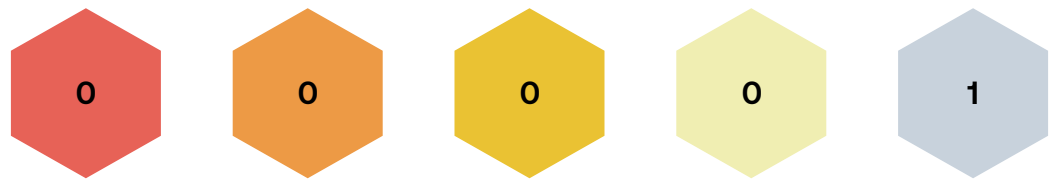
7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the Terminology section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



No Open Issues

Total number of findings



Critical | High | Medium | Low | Informational

7.1 Overview

The following table lists all identified findings along with their severity and current status. A finding is considered resolved once it has been fully corrected or the specification has been changed accordingly. Non-informational findings with any other status, including partial fixes, acknowledgements, and accepted risks, remain open.

Info	#001	`eliminateReserveDeficit` Is Not Reentrancy Guarded	Code Corrected

7.2 Descriptions

#001 `eliminateReserveDeficit` Is Not Reentrancy Guarded

Informational **Version 1** **Code Corrected**

`Pool.eliminateReserveDeficit()` is restricted by `onlyUmbrella`, but does not check or acquire the Pool's reentrancy guard. Consequently, it can execute while another guarded Pool action, such as `supply()` or `liquidationCall()`, is still in progress.

`Pool.eliminateReserveDeficit()` does not expose a callback itself, as its aToken burn does not transfer underlying tokens to an external recipient. However, another Pool action can be used as the initial entrypoint: a callback during an underlying-token transfer could trigger an authorized Umbrella call to `eliminateReserveDeficit()`. This call would pass `onlyUmbrella` and execute despite the Pool's reentrancy guard already being set. Deficit elimination would then burn Umbrella's aTokens, reduce the reserve deficit, and update reserve accounting before the outer action completes.

Umbrella itself is out of scope for this review: it can only be set by the Pool owner, which is the DAO acting behind a timelock and governance vote, so we do not consider whatever additional restrictions Umbrella may impose on who can trigger its calls.

Even assuming Umbrella's own restrictions are violated and the call is reached mid-reentrancy, we found no way for this interleaving to break accounting. The finding is therefore informational and documents a gap in the Pool's reentrancy protection.

Code Corrected

`eliminateReserveDeficit()` is now marked `nonReentrant`, with the guard check ordered ahead of `onlyUmbrella` (**Version 2**). It can no longer execute while another guarded `Pool` action is in progress.

8 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.